




# The Need for Speed: Cut Your Spring Boot Test Suite Duration by 70%

## The hidden gem of Spring Test to unlock blazing fast feedback

JUG Vilnius 2026 · October 22, 2026

Philip Riecks · @rieckpil · PragmaTech GmbH

# Let’s See Who Has the Fastest Build

📱 Scan the QR code or go to **menti.com** and enter

**1717 6052**



## About Philip

- Software Engineer from Herzogenaurach (HQ of adidas & Puma), Germany 🍻
- Blogging and content creation about testing Java and Spring Boot applications 🍃
- Founder of PragmaTech GmbH - **Enabling Developers to Frequently Deliver** Software with **More Confidence**




Herzogenaurach, Bavaria

# Why Test Software?



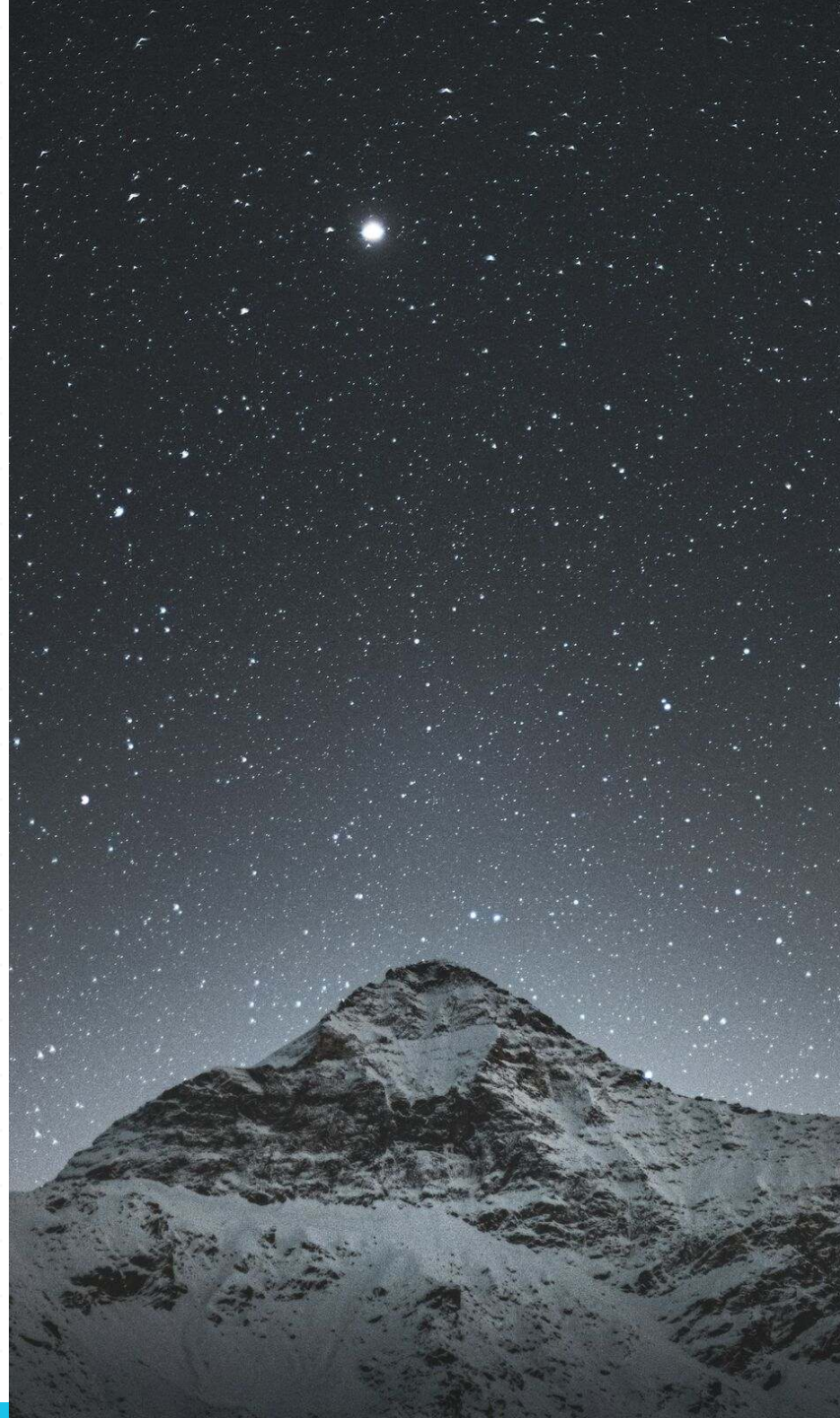
# Most of the code is generated …

# … but you still own the liability and can’t prompt confidence in your pipeline.

# How Much Testing Do We Need?

**No coverage goal**, but **engineering excellence**:

- **Confidence** in every commit
- Being able to **deploy fast** anytime
- **Fast** feedback & **fast** flow



# What We Expect From the Safety Net

## Fast

Feedback in **minutes**, not hours

## Comprehensive

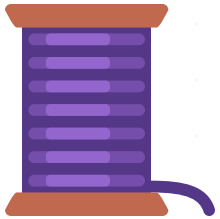
The **correct types of tests** to get confidence

# Today we focus on fast tests.



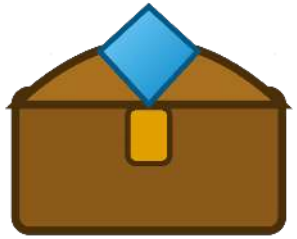
No tests at all

fastest build, zero confidence



Parallelize tests

use all your cores

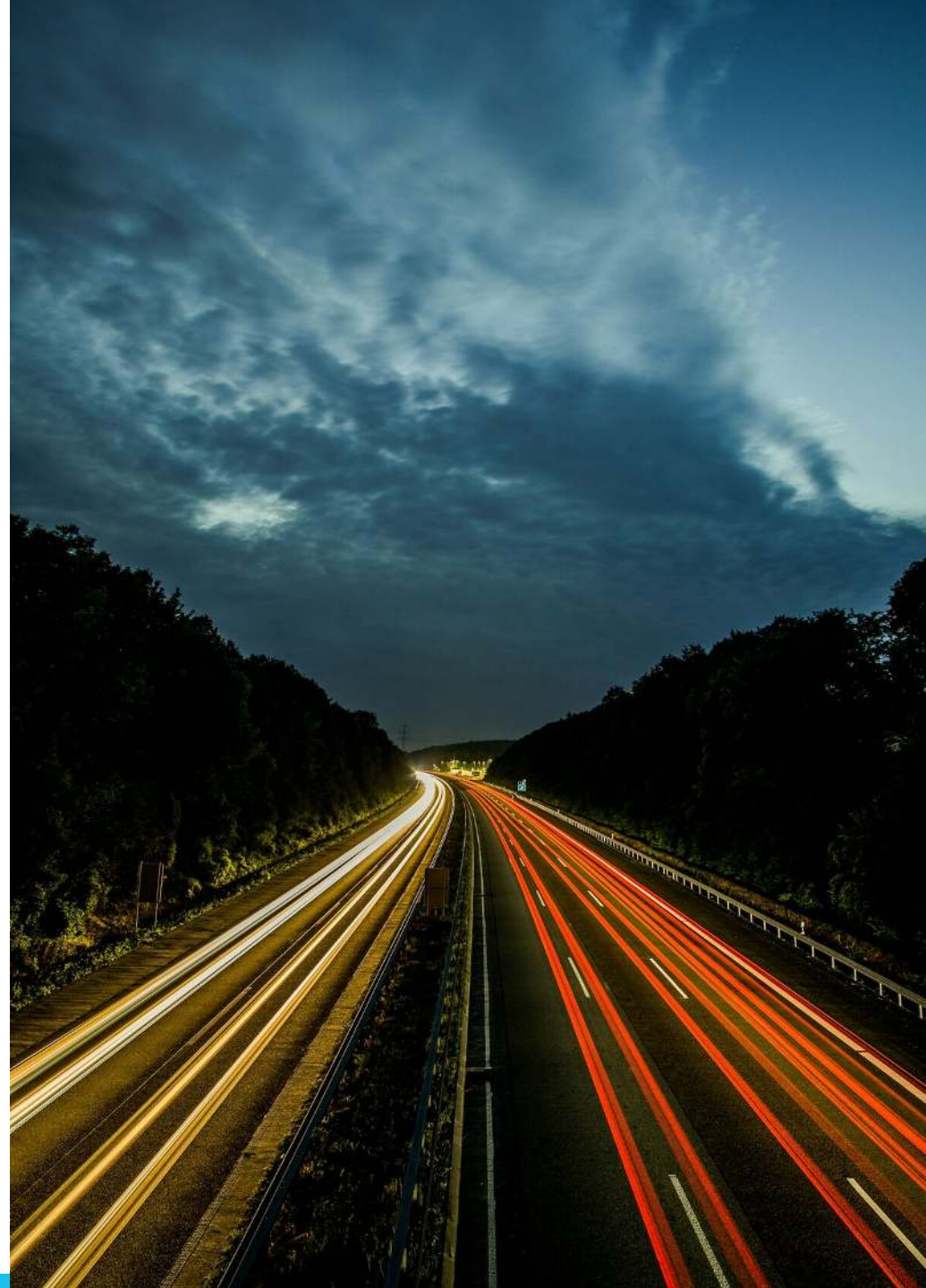


Something else ...

a treasure that helps us

THE SPRING TEST HIDDEN GEM

# Introducing: Spring TestContext Context Caching



Philip Riecks · @rieckpil · PragmaTech GmbH

# What Do These Tests Share?

```java
@WebMvcTest(BookController.class)
class BookControllerIT {

  @Autowired
  private MockMvc mockMvc;

  @MockitoBean
  private BookService bookService;

  @Test
  void shouldReturnAllBooks() {
    // ...
  }
}
```

```java
@SpringBootTest(
  webEnvironment = RANDOM_PORT)
class BookApiIT {

  @Autowired
  private TestRestTemplate template;

  @Autowired
  private BookRepository repository;

  @Test
  void shouldCreateAndFetchBook() {
    // ...
  }
}
```

# The Culprit: Every Test Starts a Context

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.1.1)

14:55:37.602 INFO  BookApiIT : Starting BookApiIT using Java 25.0.2
14:55:37.931 INFO  RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories
14:55:38.218 INFO  TomcatWebServer : Tomcat initialized with port 0 (http)
14:55:38.385 INFO  org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.4.5.Final
...
14:55:39.060 INFO  LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory
...
14:55:39.523 INFO  BookApiIT : Started BookApiIT in 2.034 seconds
```

# To Avoid: Linear Build Time Growth



# Understanding the Caching

1

Spring caches an already started context for later reuse

2

A test with a similar context configuration gets a "hot" context handed over

3

No dependency needed, it works out of the box

# How Does Spring Cache Your Context?

## Step 1: Implement a test with a context

```
@SpringBootTest
class BookApiIT {
  // ...
}
```

## Step 2: Spring analyzes your context requirements

**Profiles:** default
**Beans:** mocks, spies
**Properties:**
server.port=0
...

## Step 3: Computation of the cache key and trying to look up an existing context

**Cache hit:** Reuse hot context
**Cache miss:** Your test needs to start the context

# Each Test Requires a Different Context

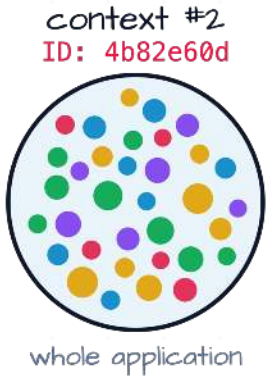
```java
@WebMvcTest(BookController.class)
class BookControllerIT {
  @MockitoBean BookService bookService;
  // ...
}
```






```java
@SpringBootTest(webEnvironment = RANDOM_PORT)
class BookApiIT {
  @Autowired TestRestTemplate template;
  // ...
}
```





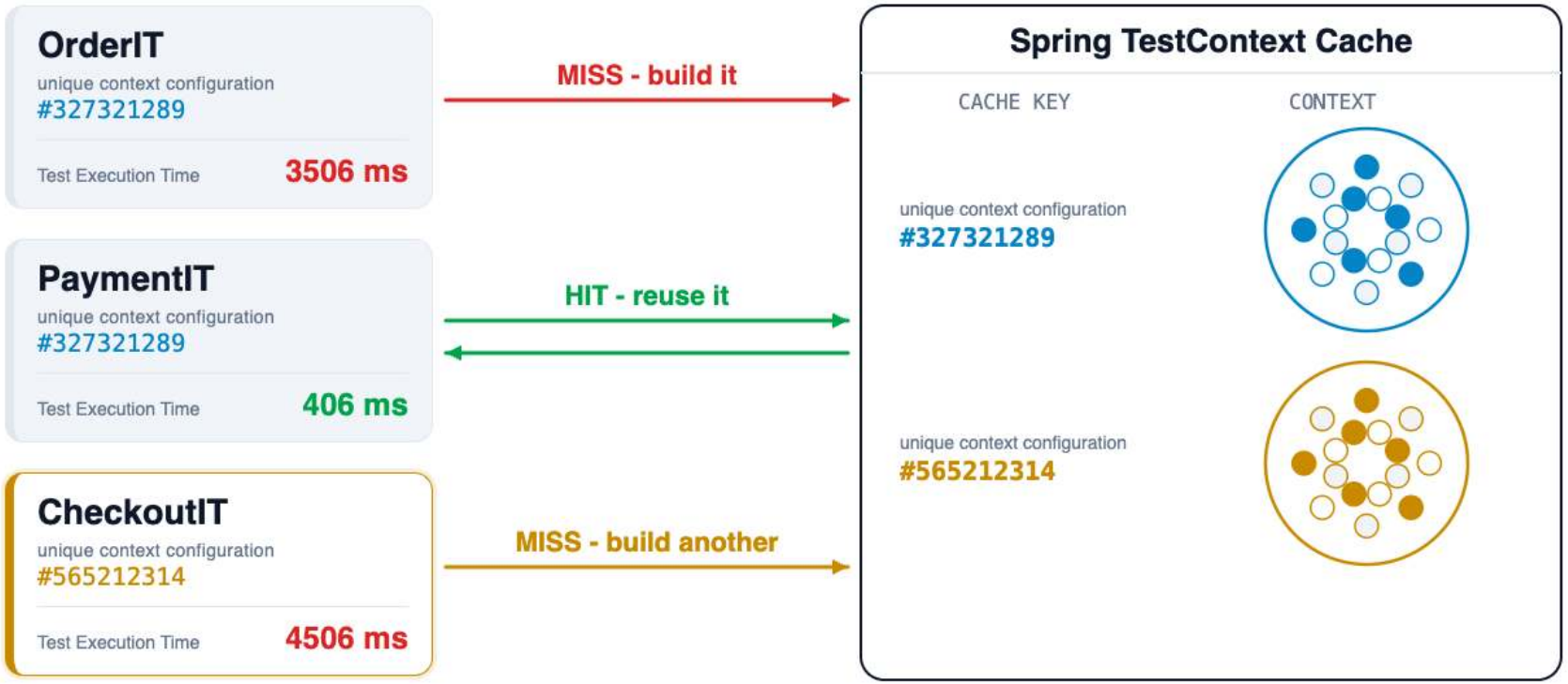
# Spring TestContext Caching Behind the Scenes

```
1   // DefaultContextCache.java
2   private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LruCache(32, 0.75f));
```

The cache key is a `MergedContextConfiguration`: **the store of all context customization points** of a test class.

- Spring collects **more than 10 things** to identify a context
- Any change results in a **new hash code** of this Java object
- A new hash code means a cache miss and a new context start

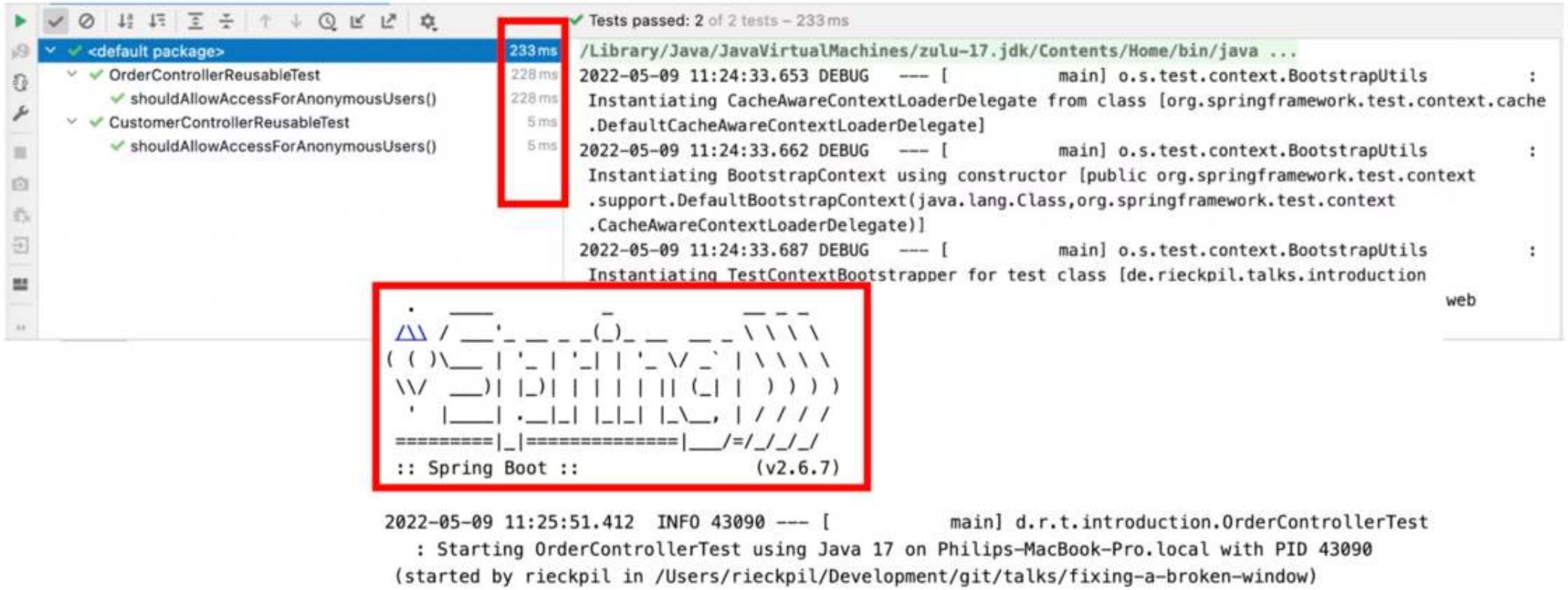
# Context Caching in Action




**Result: 3 tests ran, but 2 contexts had to start**

Only **PaymentIT** reused a cached context and finished fast - the other two each paid for a fresh one.

# Detect Context Restarts - Visually



# Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] org.springframework.test.context.cache   : Spring test
 ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
 hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [           main] tractDirtiesContextTestExecutionListener : After test class:
 context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
  = [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
 '{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
 propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
 .SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
 .filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
 .DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```

# Detect Context Restarts - with Tooling



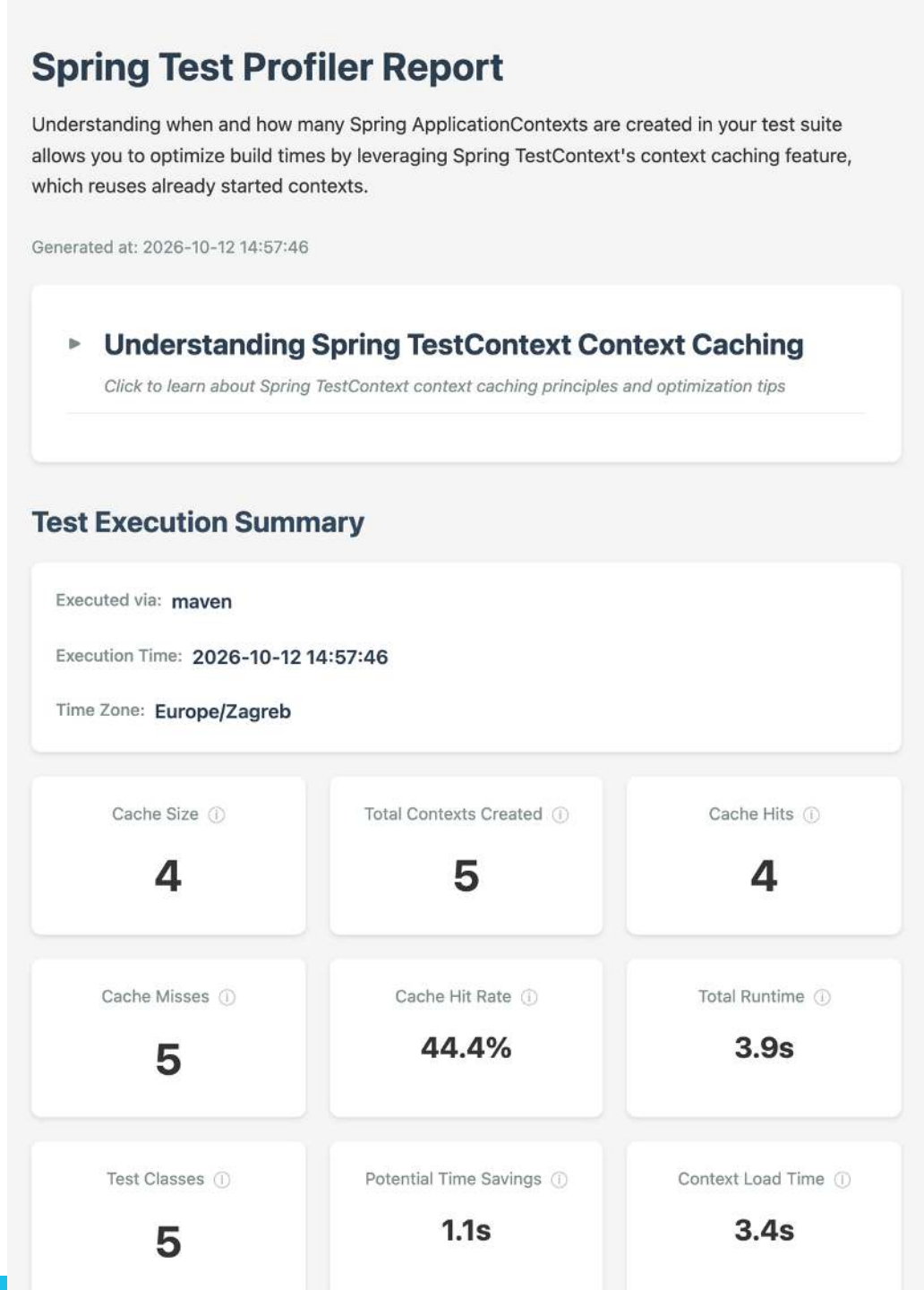

An open-source Spring Test utility that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

**Overall goal**: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.

SPRING TEST PROFILER

# Demo Time

*A quick look at the report.*



# How to Cut Build Time by 70%

1

## Understand

Measure the context starts with the Spring Test Profiler

2

## Optimize

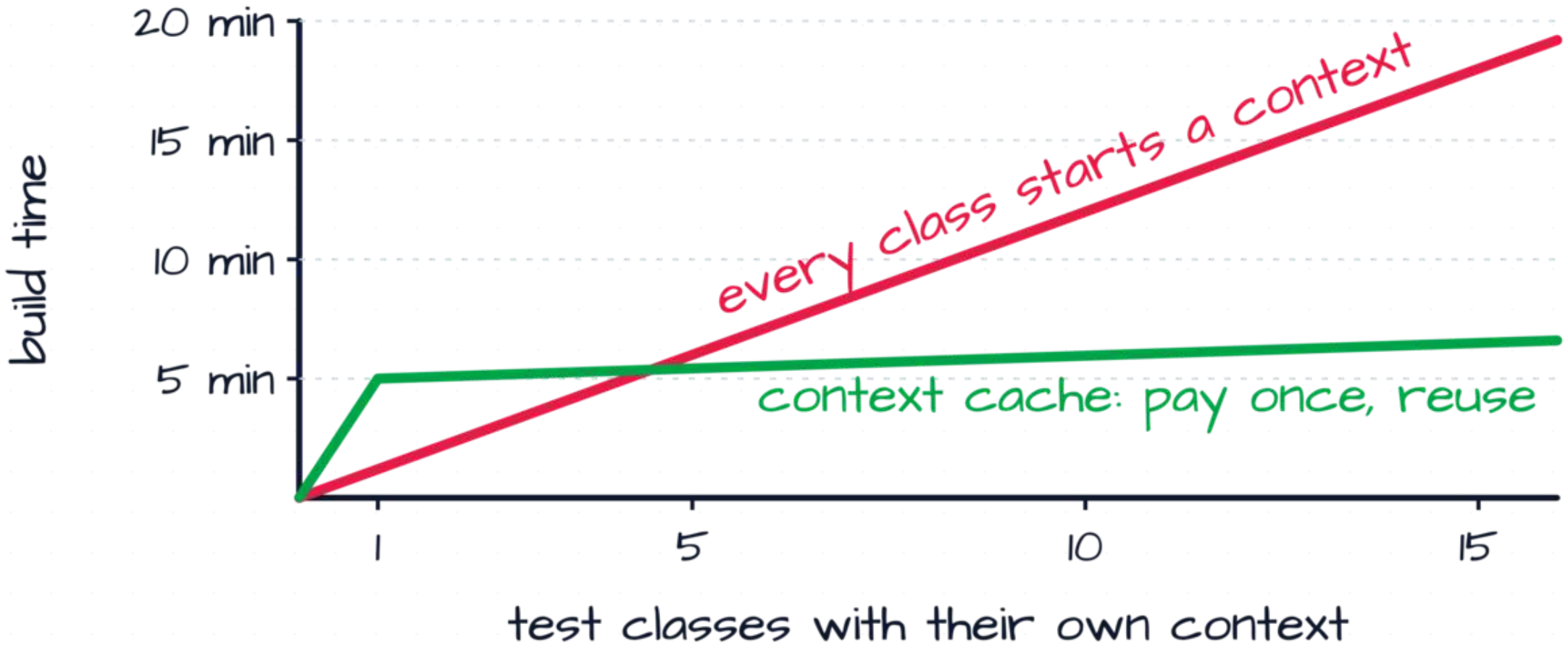
Align the configurations so contexts get reused

3

## Ensure

Keep it fast, for your team and for your AI agents

# The Goal: Steady Build Times



# Real Results: Builds I Optimized at Clients

**26 → 12 min**

build time

about 54% faster

**15 → 3 min**

build time

about 80% faster

# Add the Spring Test Profiler

An open-source Spring Test utility that visualizes context caching for your test suite:

```
<dependency>
  <groupId>digital.pragmatech.testing</groupId>
  <artifactId>spring-test-profiler</artifactId>
  <version>0.4.0</version>
  <scope>test</scope>
</dependency>
```

# Nothing to Activate

**No setup needed**: just drop the dependency and run your build.

Then open the report:

```
./mvnw verify
open target/spring-test-profiler/latest.html
```

# Don’t Customize the Context Heavily

- Every `@MockitoBean`, `@MockitoSpyBean`, `@TestPropertySource` or `@ActiveProfiles` variation is **a new context**
- Keep **one shared base class** and one shared test configuration
- Put mocks into shared configuration instead of annotating single test classes
- Slice only where it really pays off, and align the slices as well

# The Fix: One Base Class

```
1  @SpringBootTest(webEnvironment = RANDOM_PORT)
2  @AutoConfigureTestRestTemplate
3  abstract class AbstractIntegrationTest {
4  }
5
6  class BookApiIT
7      extends AbstractIntegrationTest {
8    // no own annotations anymore
9  }
```

- One annotation set for all tests
- No `@MockitoBean` or `@MockitoSpyBean`
- No extra `properties`

# The Final Boss

Developers tend to consult AI/StackOverflow for integration test issues and often copy advice from the internet without knowing the implications:

```
1   @SpringBootTest
2   @DirtiesContext
3   // this instructs Spring to remove the context from the cache
4   // and rebuild a new context on every request
5   public abstract class AbstractIntegrationTest {
6
7   }
```

The setup above will **disable** the context caching feature and slow down the builds significantly!

# Educate Your Team

- Explain the **cache key**: every new annotation combination costs a context start
- Put shared configuration into **one base class**, and review changes to it
- Treat `@DirtiesContext` and a new `@MockitoBean` like a performance decision
- Don't religiously optimize for only **one context**, a handful works

# Educate Your AI Agents Too

Agents write most of the tests now. They need the same rules:

```
.claude/skills/
├── unit-testing              fast tests without context bloat
├── slice-testing             right-sized Spring context slices
├── integration-testing       extend AbstractIntegrationTest, no new context
└── test-setup-reviewer       flags context cache killers
```

Point the skill to your `AbstractIntegrationTest` and name the antipatterns.

# The Profiler Also Writes a JSON Report

`target/spring-test-profiler/results.json` - flat and easy to consume from a pipeline:

```
{
  "schemaVersion": 1,
  "profilerVersion": "0.4.0",
  "generatedAt": "2026-10-12T12:57:46Z",
  "buildTool": "MAVEN",
  ...
  "contextsCreated": 5,
  "contextCacheHits": 4,
  "contextCacheMisses": 5,
  "contextCacheHitRatio": 0.4444,
  ...
}
```

# Let the CI/CD Pipeline Guard the Context Count

Add one step at the end of the pipeline that reads the report:

```
./mvnw verify

expectedContexts=5
actualContexts=$(jq -r '.contextsCreated' \
  target/spring-test-profiler/results.json)

if [ "$actualContexts" -gt "$expectedContexts" ]; then
  echo "Too many Spring contexts: $actualContexts"
  exit 1
fi
```

# Upcoming: Guard the Context Count with a Test

The profiler exposes the numbers to your own code (upcoming feature):

```
1   public class ContextBudgetListener implements LauncherSessionListener {
2
3     @Override
4     public void launcherSessionClosed(LauncherSession session) {
5       var statistics = SpringTestProfilerListener.getContextCacheStatistics();
6
7       if (statistics.distinctContextsCreated() > 5) {
8         throw new AssertionError("Too many Spring contexts: "
9             + statistics.describeContexts());
10      }
11    }
12  }
```

# Summary



## The problem

Every context start costs time, so build time grows with every test class



## The gem

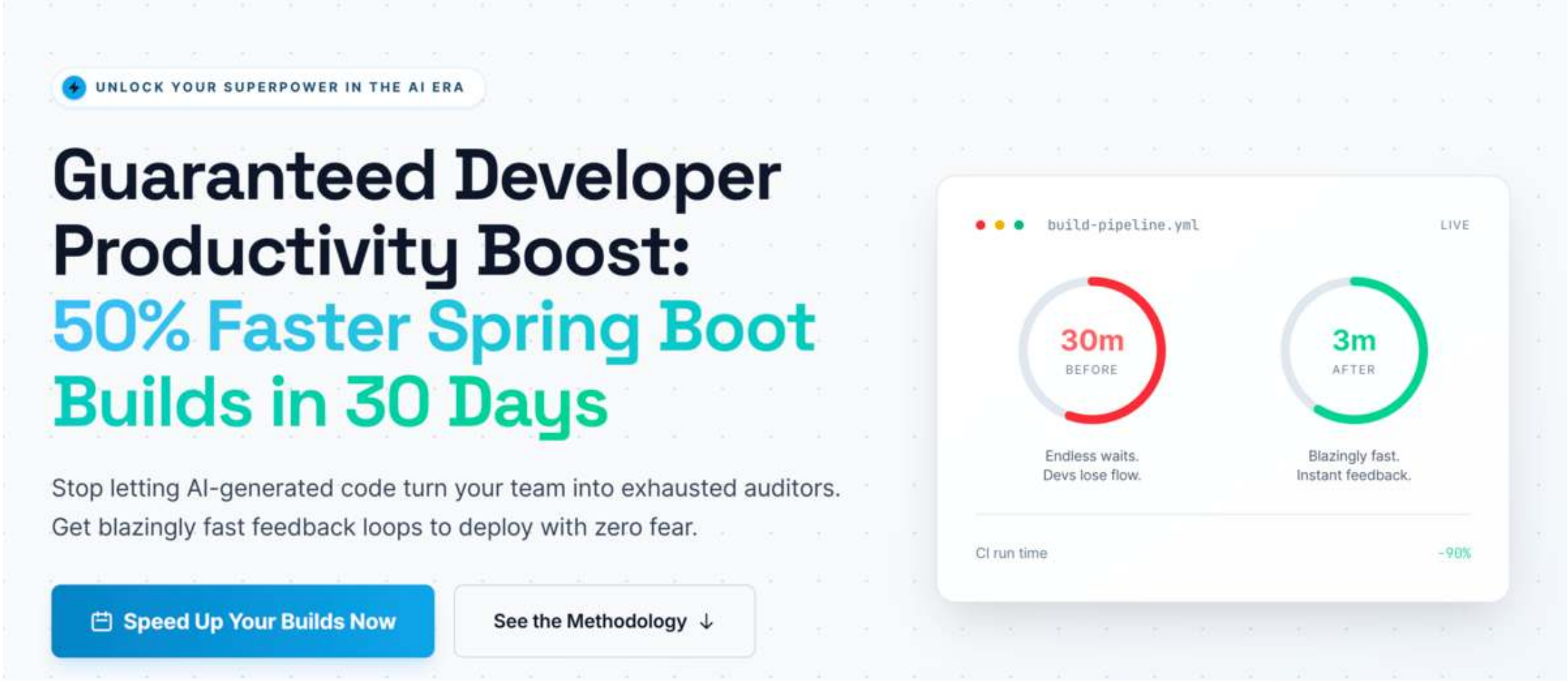
Context caching: same configuration, same context



## The plan

Understand with the profiler, optimize, ensure with CI and skills

# Need Help Further Optimizing Your Build?



# Ačiū and Joyful Testing!

Join my **Spring Boot Testing Newsletter** for best practices, recipes and quick wins in your inbox:
**rieckpil.de/newsletter**

